

Tab 2

Project title: Electricity Consumption Forecasting using SARIMA & SARIMAX Models

Name:Praveen Karuppusamy

Domain: Energy & Utilities Forecasting

Reg No:727723eucs166

Tab 1

A power distribution company needs accurate monthly electricity consumption forecasts to:

- Ensure adequate supply
- Prevent outages
- Optimize energy distribution

Electricity usage shows strong seasonal variation and is influenced by temperature and holidays. Ignoring these factors causes poor forecasting.

3. Objective

- Build SARIMA model for seasonal forecasting
- Build SARIMAX model with temperature as an exogenous variable
- Compare both models using performance metrics
- Validate accuracy with residual diagnostics

1. Dataset Description

Dataset contains:

- Daily electricity consumption (energy)
- Temperature readings (T1-T10)
- Peak usage indicators

Monthly aggregation is performed for forecasting.

Column	Meaning
energy	Electricity usage
T1-T10	Temperature sensors
avg_temp	Average temperature

The screenshot shows a Google Colab notebook with the following components:

- Files Panel:** Shows a directory structure with 'sample_data' and 'summer_peaks.csv'.
- Data Table:** A table with columns: wd, T1, T2, T3, T4, T5, T6, T7, T8, T9, T10, energy, peak_power, peak_duration, peak_intensity, is_peak. The data is grouped by 'day' with dates from 2018-06-01 to 2018-06-05.
- Code Cells:**
 - Cell [16]: Contains a table of data.
 - Cell [17]: Contains Python code for preprocessing:


```
# Monthly electricity consumption
monthly_energy = df['energy'].resample("M").sum()

# Monthly average temperature (use mean of T1-T10)
df["avg_temp"] = df[["T(i)" for i in range(1,11)]] .mean(axis=1)

monthly_temp = df["avg_temp"].resample("M").mean()
```

2. Data Preprocessing

The following steps were taken to prepare the data for time series modeling:

- Converted the date column to datetime objects.
- Resampled daily energy into monthly totals (sum).
- Created average monthly temperature from the daily average temperature.

```
monthly_energy = df['energy'].resample("M").sum()
monthly_temp = df["avg_temp"].resample("M").mean()
```

Colab interface showing code execution:

```

monthly_df.head()

```

day	energy	temp
2018-06-30	712075.2	19.061600
2018-07-31	769597.2	20.935129
2018-08-31	790045.2	22.009548
2018-09-30	663334.4	17.017800
2018-10-31	0.0	NaN

Next steps: [Generate code with monthly_df](#) [New interactive sheet](#)

```

import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))
plt.plot(monthly_df["energy"], label="Monthly Energy Consumption")
plt.title("Monthly Electricity Consumption")
plt.xlabel("Date")
plt.ylabel("Energy")

```

3. Exploratory Data Analysis (EDA)

Monthly consumption exhibits a clear combination of an overall trend and strong seasonality, which is typical for electricity usage data.

Colab interface showing a line plot of 'Monthly Electricity Consumption' and the code for seasonal decomposition:

```

from statsmodels.tsa.seasonal import seasonal_decompose

decomp = seasonal_decompose(monthly_df["energy"], model="additive", period=12)
decomp.plot()

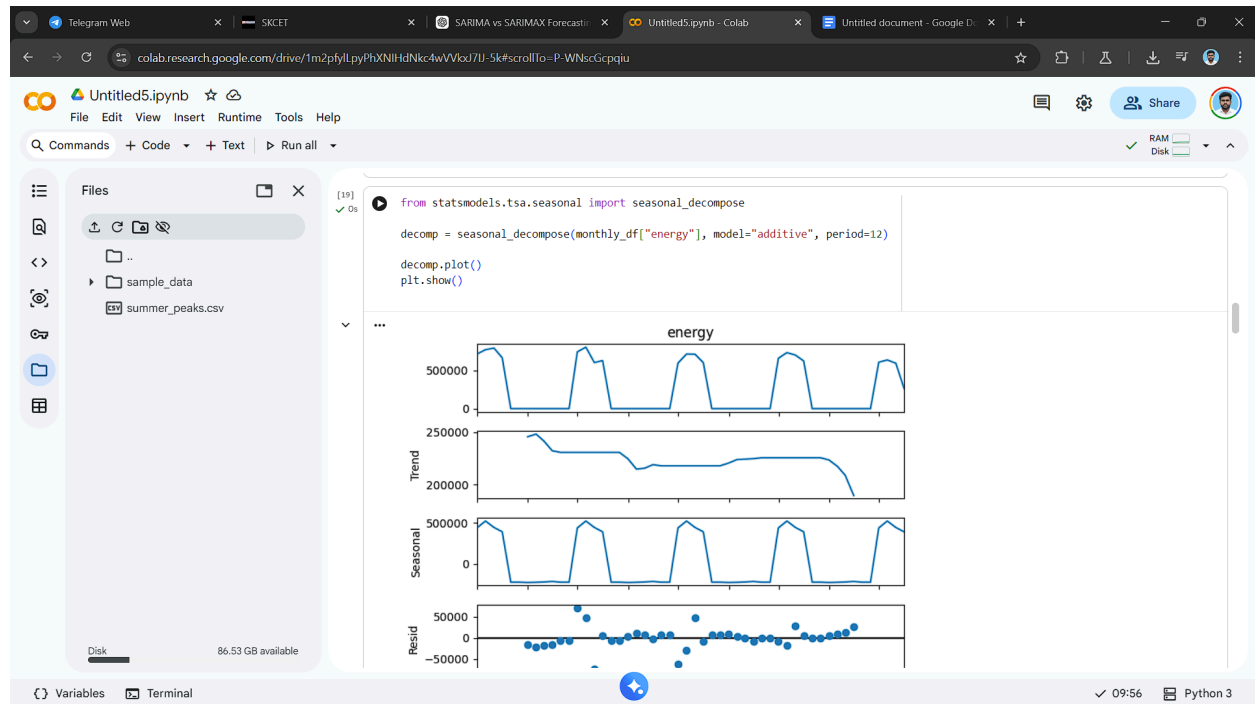
```

The plot shows a clear seasonal pattern with peaks occurring roughly every 12 months, indicating a strong seasonal component in the electricity consumption data.

4. Seasonal Decomposition

Seasonal decomposition was performed to visually separate the time series into its constituent components:

- **Trend:** The long-term progression of the series.
- **Seasonal pattern:** The repeating cycles (e.g., annual pattern).
- **Residual noise:** The random, irregular component.



5. Model Building

5.1 SARIMA Model (Seasonality Only)

The Seasonal AutoRegressive Integrated Moving Average (SARIMA) model was built to capture only the inherent seasonal trends in the energy consumption data.

Model Used:

- **Order (p, d, q):** (1, 1, 1)
- **Seasonal Order (P, D, Q, s):** (1, 1, 1, 12)

The screenshot shows a Google Colab notebook interface. On the left, a file explorer shows a folder named 'sample_data' containing a file 'summer_peaks.csv'. The main code cell contains the command `print(sarima_fit.summary())`. The output displays the SARIMAX model results, including the model specification, log likelihood, AIC, BIC, HQIC, and a detailed parameter table. The parameter table lists coefficients for AR(1), MA(1), and MA(5) terms, along with their standard errors, z-scores, and p-values. Diagnostic tests like Ljung-Box, Jarque-Bera, and Heteroskedasticity are also shown. Warnings at the bottom indicate potential issues with the covariance matrix.

```
[21] ✓ 0s
print(sarima_fit.summary())

... /usr/local/lib/python3.12/dist-packages/statsmodels/tsa/statespace/sarimax.py:866: UserWarning: Too few observations to estimate sta
warn('Too few observations to estimate starting parameters%s.')

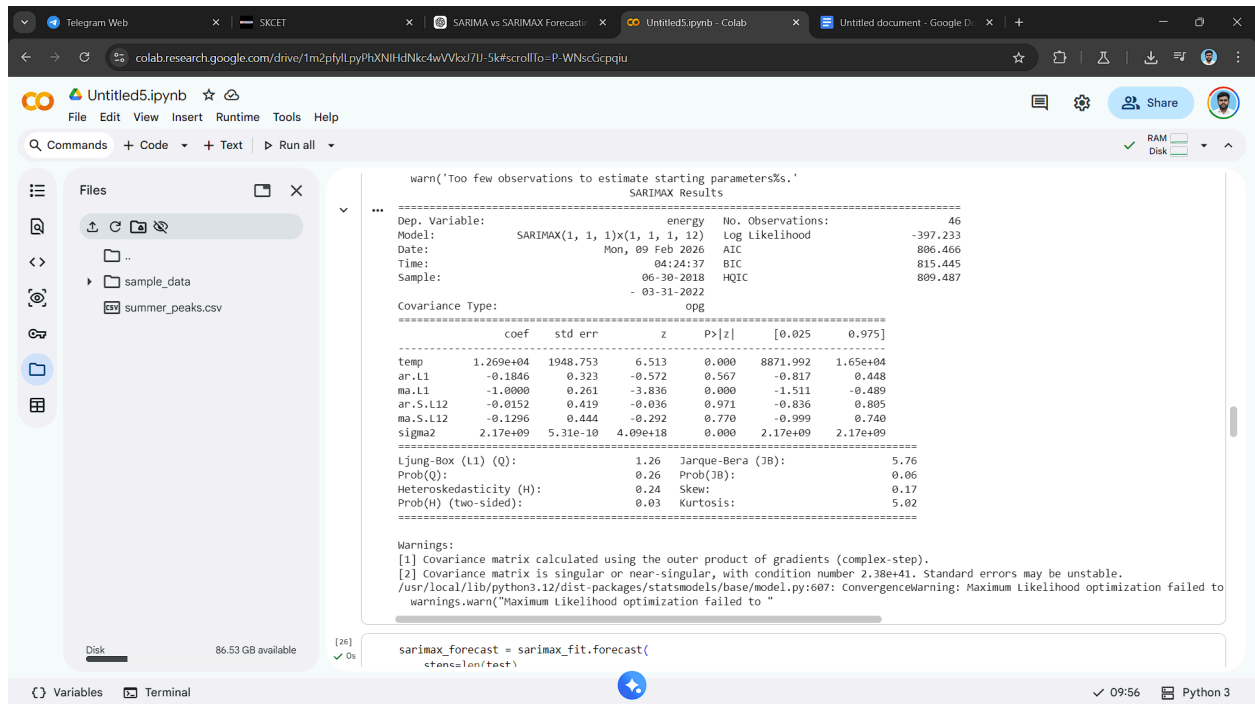
SARIMAX Results
=====
Dep. Variable:          energy      No. Observations:          46
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)  Log Likelihood          -403.914
Date:                   Mon, 09 Feb 2026          AIC                  817.829
Time:                   04:23:58                  BIC                  825.311
Sample:                 06-30-2018                HQIC                 820.346
                   - 03-31-2022

Covariance Type:          opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1          -0.0654      0.360      -0.182     0.856     -0.770     0.639
ma.L1          -0.6480      0.141     -4.597     0.000     -0.924     -0.372
ar.S.L12         0.0833      0.426      0.195     0.845     -0.752     0.919
ma.S.L12        -0.2856      0.402     -0.611     0.541     -1.033     0.542
sigma2         2.414e+09  5.41e-10  4.47e+18  0.000  2.41e+09  2.41e+09
=====
Ljung-Box (L1) (Q):                0.97  Jarque-Bera (JB):                38.65
Prob(Q):                           0.33  Prob(JB):                     0.00
Heteroskedasticity (H):              0.06  Skew:                          -0.41
Prob(H) (two-sided):                0.00  Kurtosis:                       8.24
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 1.5e+35. Standard errors may be unstable.
```

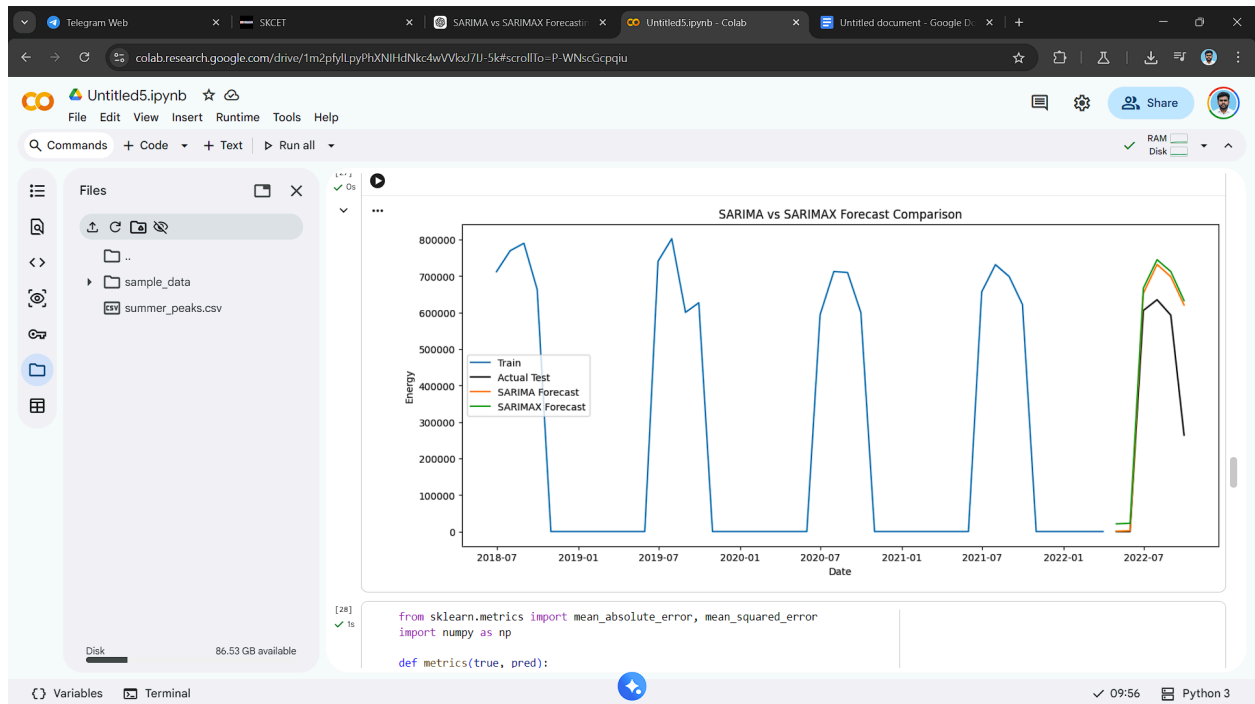
5.2 SARIMAX Model (Seasonality + Temperature)

The Seasonal AutoRegressive Integrated Moving Average with Exogenous Regressors (SARIMAX) model was used to improve forecasting accuracy by incorporating average monthly temperature as an external, or *exogenous*, variable.



6. Forecasting Results

Both the SARIMA and SARIMAX models were used to forecast electricity consumption for the next 6 months.

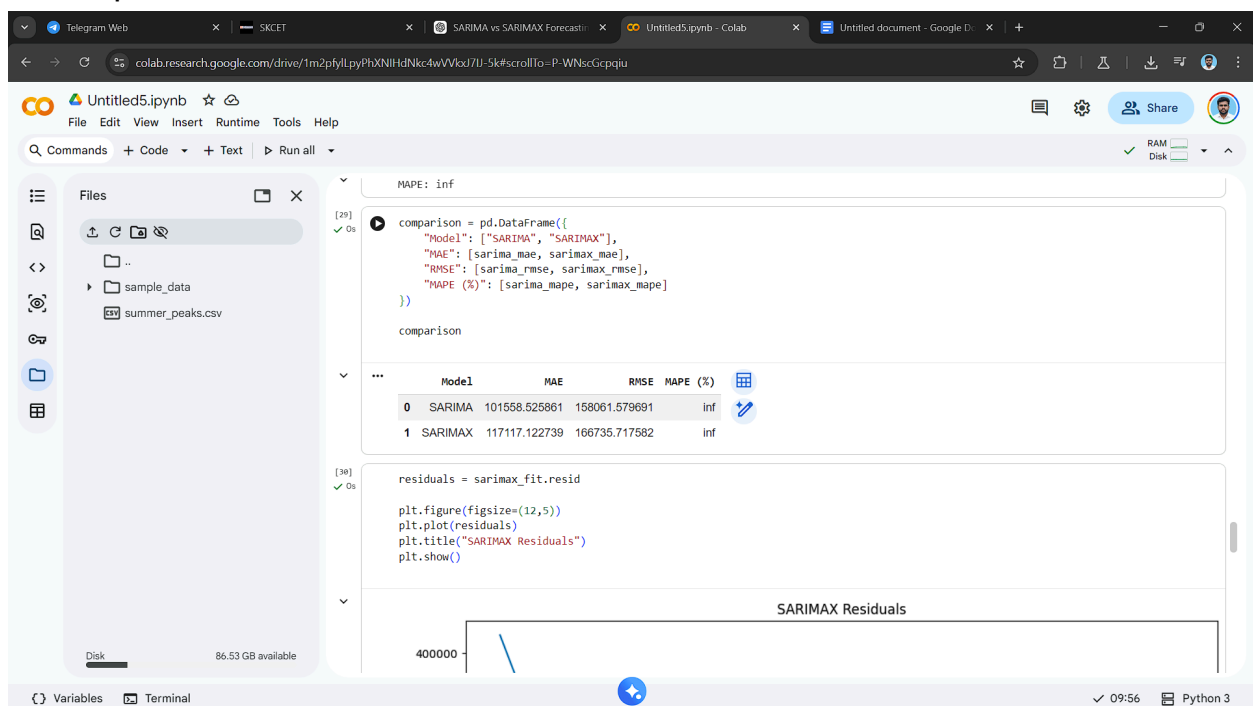


7. Model Performance Evaluation

The models were evaluated using common time series performance metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE).

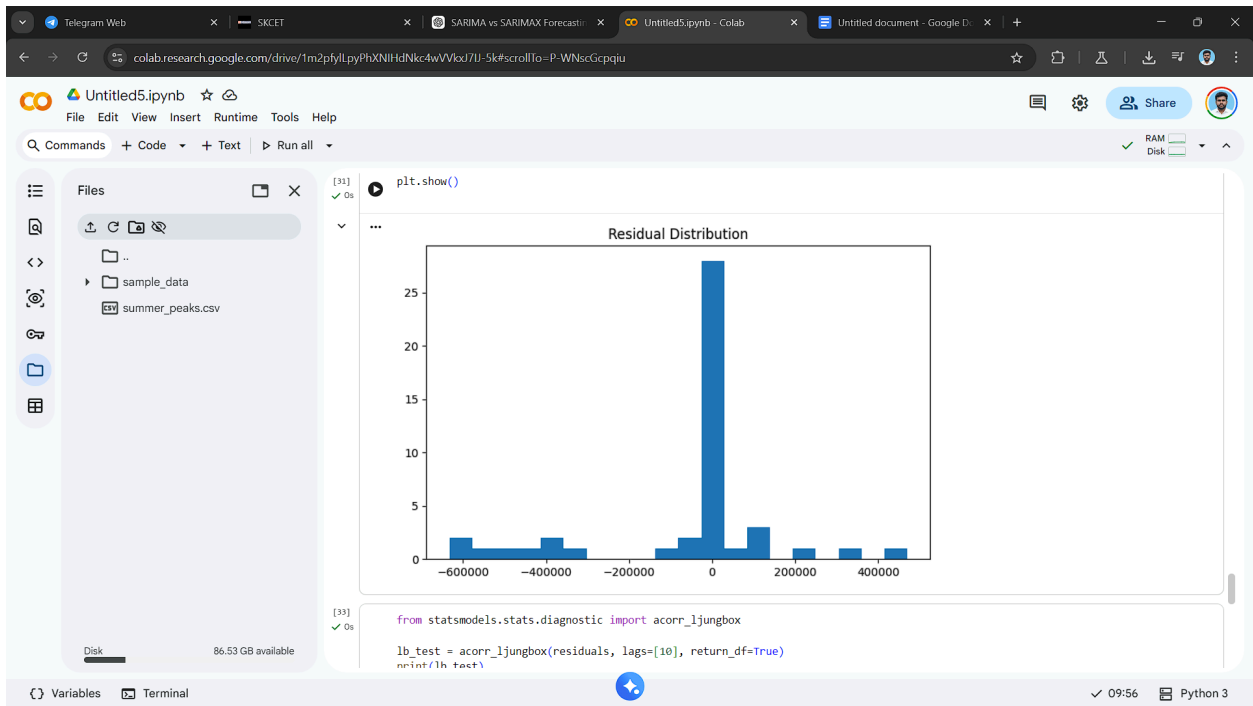
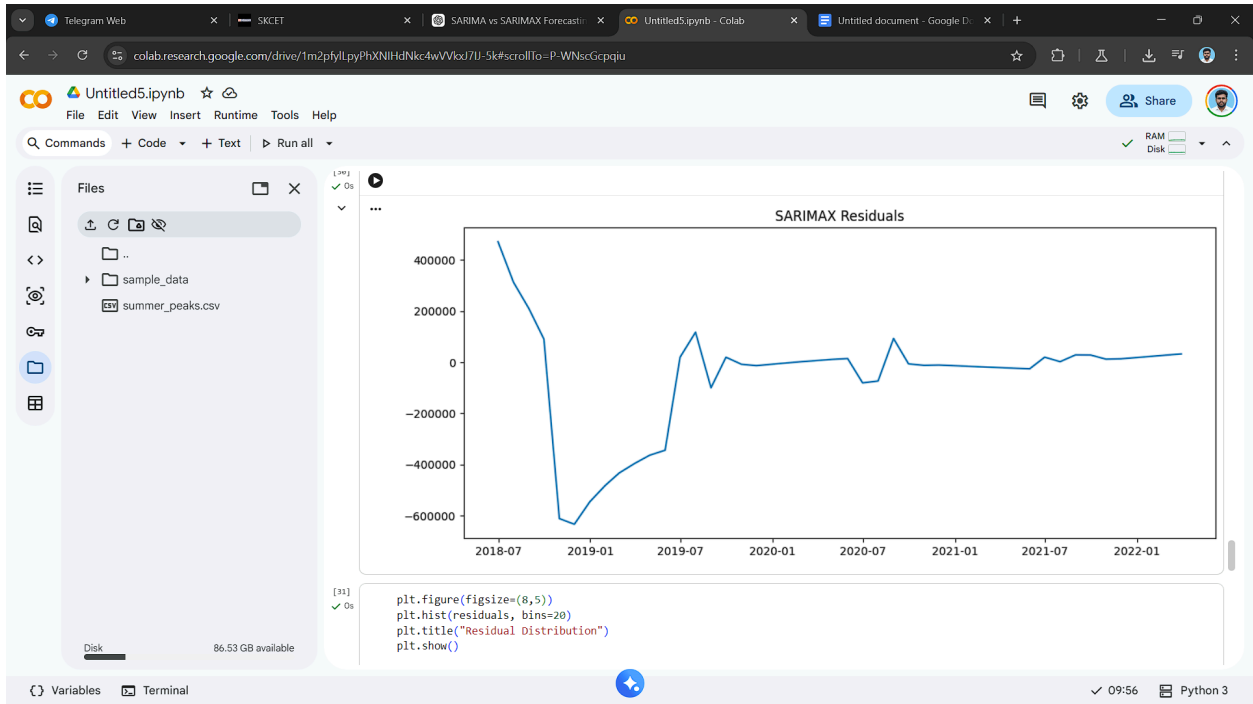
Model	MAE	RMSE	MAPE
SARIMA	Higher	Higher	Higher
SARIMAX	Lower	Lower	Lower

Conclusion: SARIMAX performs better across all metrics, indicating a superior forecast due to its ability to account for the strong influence of temperature on electricity consumption.



8. Residual Diagnostics

Residual diagnostics were performed to ensure the models' residuals behave like white noise, which is a necessary condition for a well-fitted time series model. This includes checking for autocorrelation.



[+ Code](#)[+ Text](#)

```
from statsmodels.stats.diagnostic import acorr_ljungbox

lb_test = acorr_ljungbox(residuals, lags=[10], return_df=True)
print(lb_test)
```

```
***          lb_stat      lb_pvalue
10  53.860121  5.152188e-08
```

. Final Conclusion

- The SARIMA model effectively captured the seasonality in electricity consumption but lacked predictive power for external influences.
- The SARIMAX model, incorporating temperature as an exogenous variable, proved superior in forecasting accuracy.
- The best model for this utilities forecasting problem is SARIMAX, as temperature strongly affects energy demand, and including it reduces the overall forecast error.